



AYRON SANFRA SILVA MARINHO

IGOR ABREU FORTUNATO

LUIZA OLIVEIRA DE SOUZA GARCIA

DIAGRAMA DE CLASSES E ESTRUTURA MVC

JUBA - SOFTWARE PARA CONTROLE FINANCEIRO PESSOAL

LAVRAS - MG
2024

AYRON SANFRA SILVA MARINHO

IGOR ABREU FORTUNATO

LUIZA OLIVEIRA DE SOUZA GARCIA

DIAGRAMA DE CLASSES E MVC

JUBA - SOFTWARE PARA CONTROLE FINANCEIRO PESSOAL

Documento apresentado ao curso de Engenharia de Software como requisito parcial para aprovação na disciplina. Orientado pelo professor Heitor Augustus Xavier Costa.

LAVRAS - MG
2024

LISTA DE FIGURAS

Figura 01 - Diagrama de Classes

Figura 02 - Tecnologias na Estrutura MVC

Figura 03 - Estrutura MVC

SUMÁRIO

INTRODUÇÃO	8
DIAGRAMA DE CLASSES	9
DESCRIÇÃO DO DIAGRAMA DE CLASSES	10
ESTRUTURA MVC COM TECNOLOGIAS	12
DESCRIÇÃO DA ESTRUTURA MVC	15
REFERÊNCIAS	16

INTRODUÇÃO

Um diagrama de classes é uma representação visual da estrutura de um sistema de software, detalhando as classes, seus atributos, métodos e relacionamentos entre elas. No contexto de um software de gerenciamento financeiro, como o Juba, um diagrama de classes é uma ferramenta crucial para projetar e desenvolver um sistema funcional. Ele ajuda a identificar as entidades principais, como usuários, transações e categorias, e a definir como essas entidades se relacionam entre si. Além disso, o diagrama de classes permite modelar os diferentes tipos de transações, as categorias disponíveis e os relatórios que podem ser gerados, garantindo que o software atenda às necessidades específicas dos usuários no controle de suas finanças pessoais.

Juntamente com o Diagrama de Classes, o padrão de arquitetura *Model-View-Controller* (MVC) é amplamente utilizado no desenvolvimento de software, especialmente em aplicações web. Este padrão organiza o código em três componentes principais, cada um com responsabilidades distintas, facilitando a manutenção, a escalabilidade e a reutilização de código. Um diagrama MVC ilustra como esses componentes interagem entre si para fornecer uma aplicação funcional e modular.

DIAGRAMA DE CLASSES

A Figura 01 a seguir apresenta o diagrama de classes elaborado para o software de controle financeiro “Juba”.

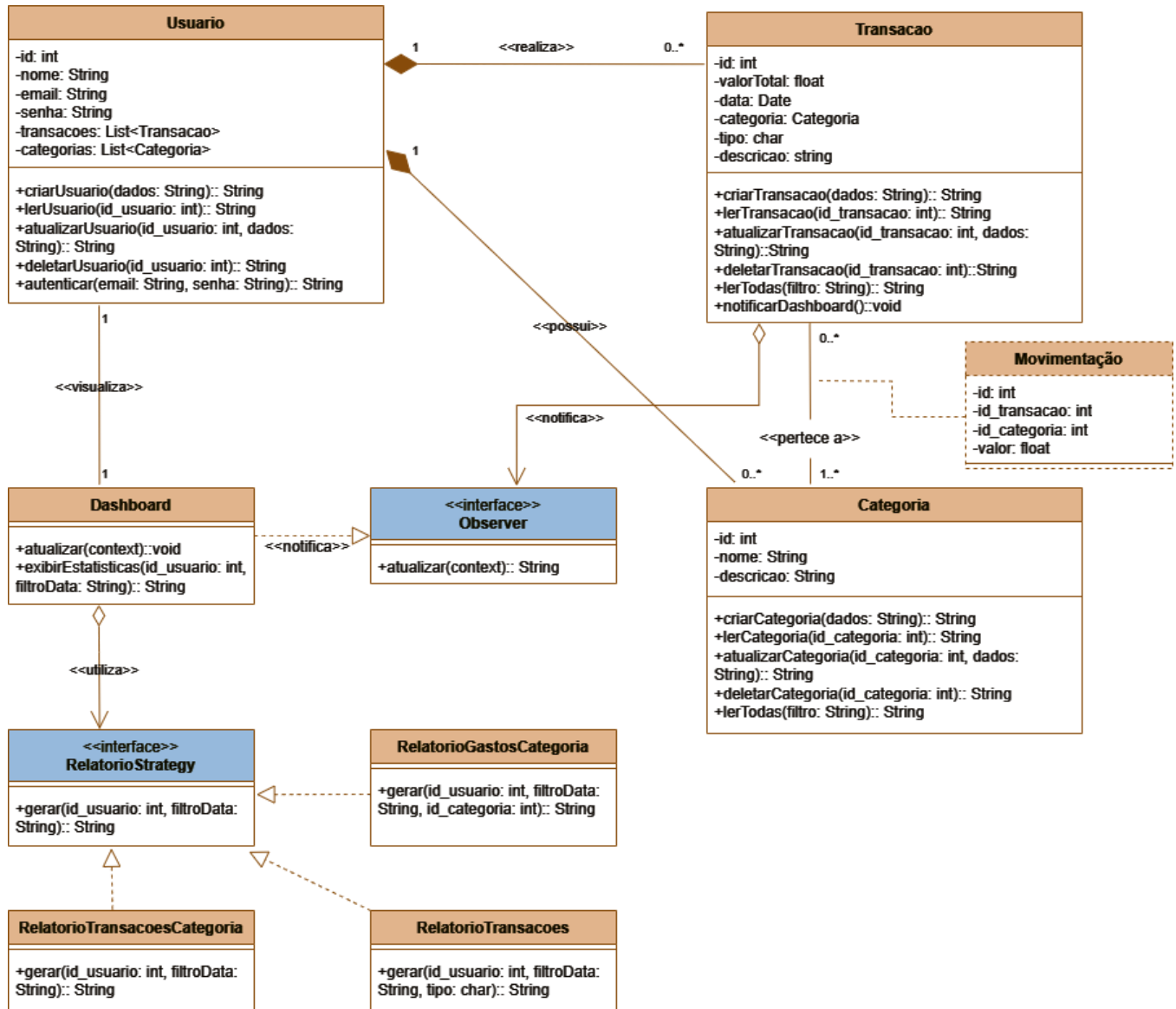


Figura 01 - Diagrama de Classes

DESCRIÇÃO DO DIAGRAMA DE CLASSES

A classe 'Usuario' é responsável pela gestão dos usuários da plataforma, armazenando informações como nome, email, senha, atributo de identificação, transações e categorias associadas. Além dos atributos, a classe conta com métodos que retornam *Strings* ao realizar operações de CRUD (*Create, Read, Update, Delete*), pois estruturas JSON (*JavaScript Object Notation*), formatação utilizada para estruturar dados em formato de texto, serão retornadas ao front-end para manipulação das informações. Um método de autenticação valida o acesso ao sistema, garantindo a segurança das informações pessoais dos usuários.

A classe 'Transação' tem como objetivo gerenciar as principais informações financeiras inseridas pelos usuários, como valor total, data e tipo (campo que o usuário poderá utilizar para classificar a transação como despesa ou receita), além de atributos de identificação incluídos pelo sistema. Esta classe oferece métodos que retornam *Strings* ao cadastrar, atualizar, deletar e listar transações. Os métodos permitirão que o *software* proporcione aos usuários um controle completo de suas finanças.

A classe 'Categoria' é responsável pela gestão das categorias disponíveis para associação às transações. Ela possui métodos CRUD, o que permite aos usuários organizar e classificar suas transações de acordo com suas preferências e necessidades. A listagem de todas as categorias permite a visualização e seleção eficiente durante a gestão financeira.

A interface 'RelatorioStrategy' define um método para a geração de relatórios. Ela é implementada por várias classes de relatório, permitindo a aplicação do padrão de projeto *Strategy* para a geração de diferentes tipos de relatórios financeiros. As classes 'RelatorioTransacoes', 'RelatorioTransacoesCategoria' e 'RelatorioGastosCategoria' especificam o método *gerar()* retornando à interface o relatório escolhido pelo usuário. A utilização do padrão *Strategy* permite a simplificação da geração de relatórios uma vez que teremos uma interface única para se comunicar com as classes específicas de geração.

A interface 'Observer' define o método para atualização e faz parte do padrão de projeto *Observer*. Esse padrão é utilizado para estabelecer uma dependência um-para-muitos entre objetos, de forma que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente. No

contexto deste sistema financeiro, a interface Observer é utilizada para notificar o Dashboard sobre mudanças nas transações.

A classe 'Dashboard' implementa a interface 'Observer' e é responsável por exibir dados dinâmicos baseados no usuário logado. Esta classe observa as mudanças nas transações e categorias e reage a essas mudanças atualizando sua interface e exibindo estatísticas financeiras relevantes.

As relações de composição utilizadas para os pares 'Transacoes' → 'Usuario' e 'Categoria' → 'Usuario', asseguram a o gerenciamento e controle desses itens pelo usuário dentro do sistema.

As relações de implementação utilizadas entre as classes específicas de relatório, sendo elas '*RelatorioTransacoes*', '*RelatorioTransacoesCategoria*' e '*RelatorioGastosCategoria*', juntamente com a interface 'RelatorioStrategy' e o par 'Observer' permitem a simplificação e maior eficiência do sistema por meio da utilização de padrões projetos já consolidados no mercado.

ESTRUTURA MVC COM TECNOLOGIAS

A Figura 02 a seguir apresenta as tecnologias utilizadas em cada parte do sistema, seguindo a estrutura MVC.

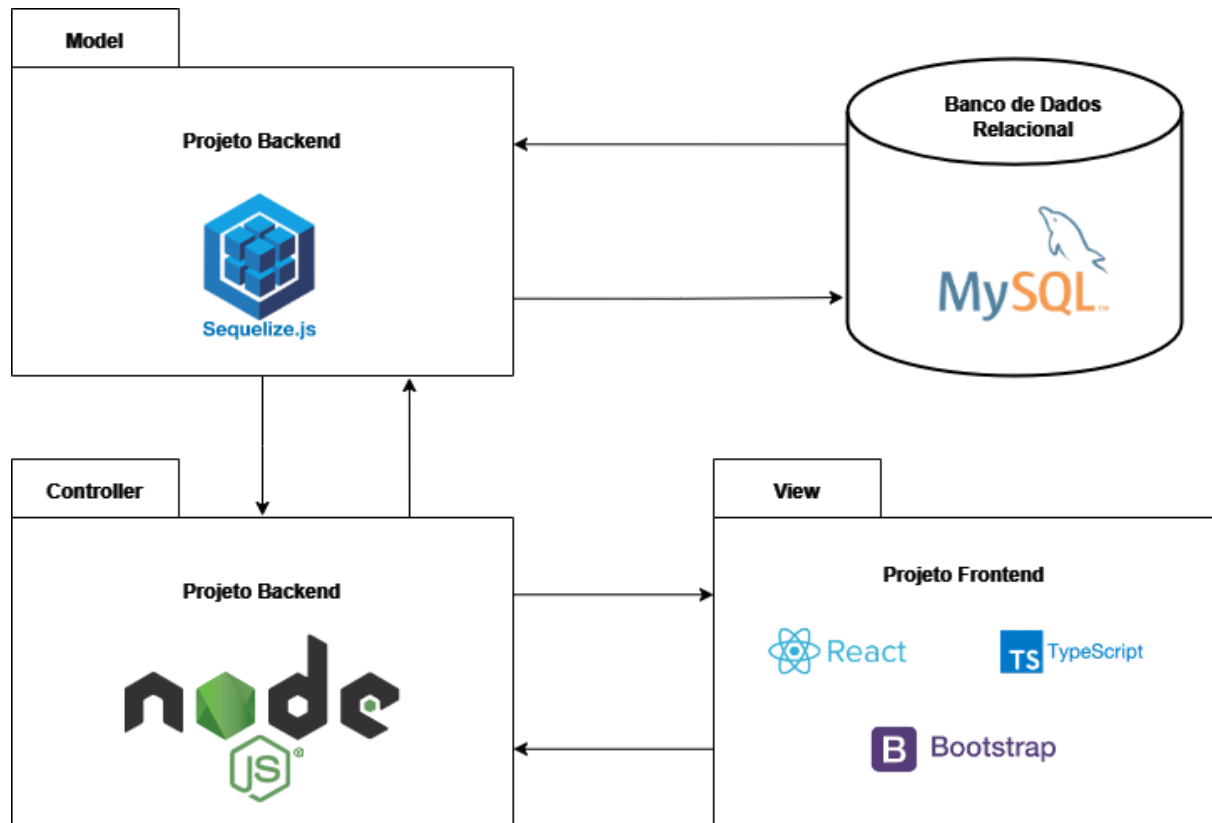


Figura 02 - Tecnologias na Estrutura MVC

DESCRIÇÃO DAS TECNOLOGIAS NA ESTRUTURA MVC

O componente *Model* representa a lógica de negócios e os dados da aplicação. Ele gerencia o acesso e a manipulação dos dados, interagindo com o banco de dados. O *Model* notifica o *Controller* sobre as mudanças nos dados, garantindo que a interface do usuário esteja sempre atualizada. Esse componente será implementado em nosso projeto utilizando a tecnologia de gerenciamento de dados *MySQL* devido à sua fácil manipulação, segurança e compatibilidade com o restante do sistema.

O componente *View* é responsável pela apresentação dos dados ao usuário. Ele exibe a interface gráfica e recebe as entradas do usuário. A *View* é indiretamente associada ao *Model*, passando antes pelo processamento da *Controller* para refletir as mudanças nos dados em tempo real, proporcionando uma experiência de usuário intuitiva e responsiva. No projeto, será utilizado a tecnologia *React.js* por ser um framework consolidado no mercado e oferecer alta flexibilidade à estruturação do projeto, permitindo a reutilização e fácil manutenção de código por meio da componentização de elementos. Além disso, será utilizada a linguagem *Typescript* que proporciona uma menor margem de risco e falhas, já que possui uma estrutura básica de tipagem de dados. Será utilizada também a biblioteca de estilos *Bootstrap* para maior eficiência e padronização na estilização da interface.

O componente *Controller* atua como intermediário entre o *Model* e a *View*. Ele processa as entradas do usuário, invocando métodos no *Model* para modificar os dados e atualizar a *View*. O *Controller* interpreta as ações do usuário e decide como responder a essas ações, gerenciando a lógica de fluxo da aplicação. O *Controller* será representado pelo nosso projeto *back-end* e utilizará a tecnologia *Node.js* o que traz as seguintes vantagens: Maior compatibilidade com o projeto *front-end*, uma vez que este utilizará uma linguagem baseada em *JavaScript*, fácil manutenção e escalabilidade, além de documentações e suporte de comunidade mais completos e consolidados.

ESTRUTURA MVC

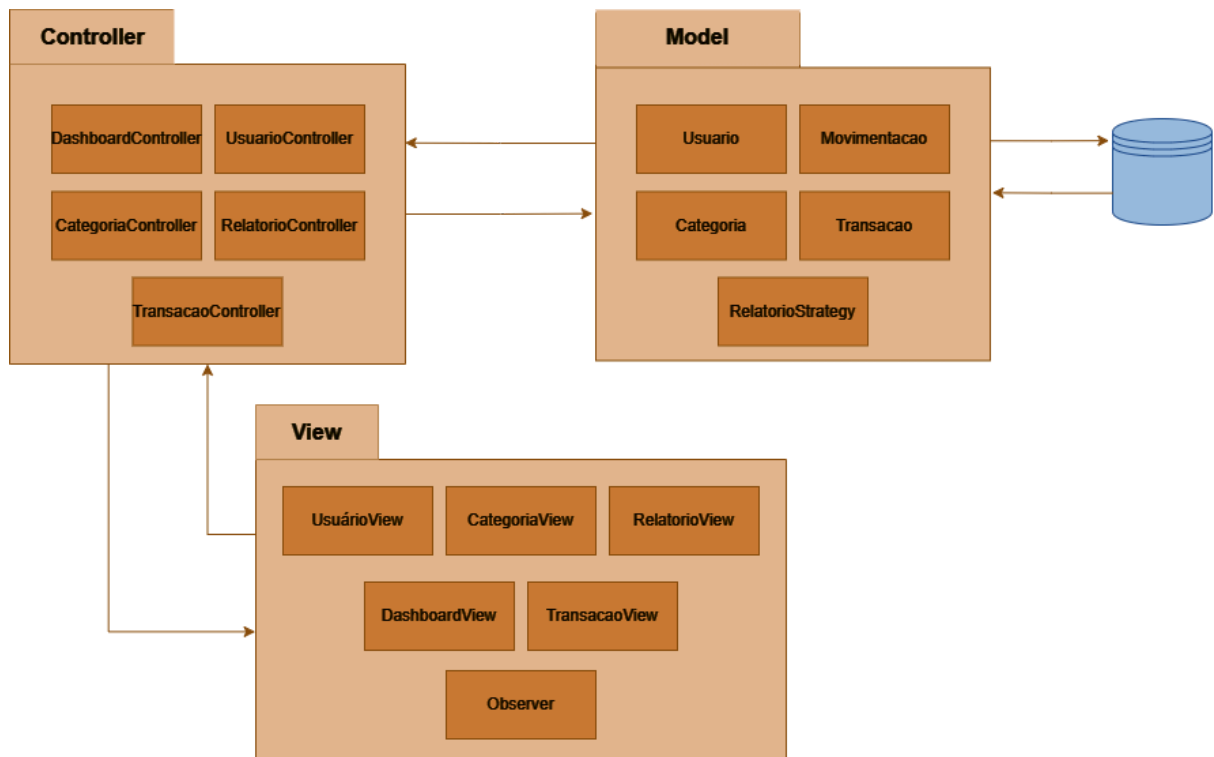


Figura 03 - Estrutura MVC

DESCRIÇÃO DA ESTRUTURA MVC

No contexto do sistema de gestão financeira pessoal, com os *Models* representando os dados e a lógica de negócios do sistema, as classes que estarão presentes nessa parte da estrutura são: 'Usuário', 'Transação', 'Categoria', 'RelatorioStrategy' e 'Movimentação' (sendo responsável por armazenar o atributo valor e decorrente da associação de Categoria com Transação).

As *Views* existentes são: 'UsuarioView', 'CategoriaView', 'RelatorioView', 'DashboardView' e 'TransaçãoView'. A 'UsuarioView' exibe e gerencia a interface relacionada aos usuários. A 'CategoriaView' exibe e gerencia a interface relacionada às categorias. A 'RelatorioView' apresenta os diferentes tipos de relatórios financeiros. A 'DashboardView' exibe as estatísticas financeiras e dados dinâmicos com base no usuário logado. Por fim, a 'TransaçãoView' exibe e gerencia a interface das transações. A interface 'Observer' atualiza a interface do Dashboard com base nas mudanças nas transações.

No sistema, as classes de *Controller* incluem 'DashboardController', 'UsuarioController', 'CategoriaController', 'RelatórioController' e 'TransacaoController'. O 'DashboardController' gerencia a exibição dos dados dinâmicos no Dashboard. O 'UsuarioController' lida com as operações CRUD relacionadas aos usuários. O 'CategoriaController' gerencia as categorias e suas operações CRUD. O 'RelatórioController' controla a geração e exibição dos diferentes relatórios financeiros. Finalmente, o 'TransacaoController' lida com as operações CRUD relacionadas às transações.

Essa estrutura MVC permite uma clara separação de responsabilidades, onde os *Models* gerenciam os dados e a lógica de negócios, as *Views* cuidam da apresentação e interação com o usuário, e os *Controllers* servem como intermediários, manipulando dados e atualizando a interface conforme necessário. Isso resulta em um sistema organizado, modular e fácil de manter e expandir.

REFERÊNCIAS

1. REIS, Fábio dos. Curso de UML - O que é um Diagrama de Classes . [Vídeo]. YouTube. Publicado em 15 de novembro de 2018. Disponível em: <[Curso de UML - O é um diagrama de classes](#)>. Acesso em: 24 de abril de 2024.
2. REIS, Fábio dos. Curso de UML - Diagrama de Classes - Relacionamentos . [Vídeo]. YouTube. Publicado em 15 de novembro de 2018. Disponível em: <[Curso de UML - Diagrama de Classes - Relacionamentos](#)>. Acesso em: 24 de abril de 2024.